

Programação Orientada a Objetos

Prof. Dr. Josenalde Barbosa de Oliveira

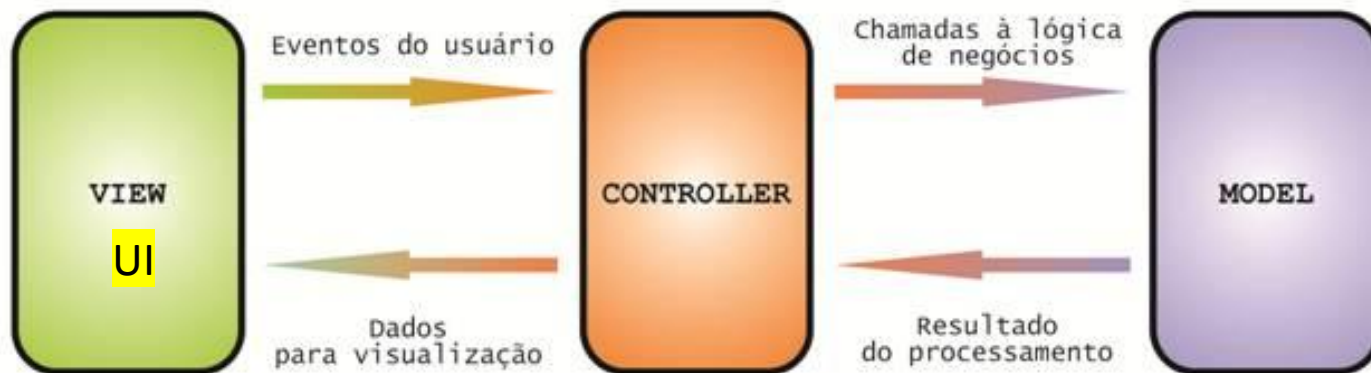
josenalde.oliveira@ufrn.br

<https://github.com/josenalde/poo>

MODEL-VIEW-CONTROLLER

Arquitetura de software

- Ideia de Trygve Reenskaug (Xerox PARC) em 1979 descrita em 1987 e depois 1992 por Steve Burbeck para projetos na linguagem SmartTalk-80. Linguagem com primeira versão em 1972 que introduziu na versão **80** conceitos de **POO** usados até hoje, incluindo o padrão MVC
- Por comparação, Java 1.0 surge em 1996 (Sun Microsystems), que é adquirida pela Oracle em 2008
- Ideia: 3 camadas independentes para reduzir acoplamento (dependências) e aumentar coesão (finalidade bem especificada) das classes do projeto



Fonte: <https://www.devmedia.com.br/padrao-mvc-java-magazine/21995>

MODEL-VIEW-CONTROLLER

Arquitetura de software

- No MODEL poderemos ter também o padrão **Data Access Object** (DAO) para separar classes de acesso ao Banco de Dados das classes de Regras de Negócio (RN, lógica). (Para se aprofundar ver também a **API JPA/Hibernate e conceito ORM**)
- No MVC isolam-se regras de negócio da UI, permitindo UIs diferentes sem alterar as RN
- Pode ser aplicado em desktop, web, mobile...
- Comunicação entre UI e regras de negócio se dá pelas classes controladoras
- Model possui a lógica da aplicação: RNs, persistência/crud e classes de entidades

MODEL-VIEW-CONTROLLER

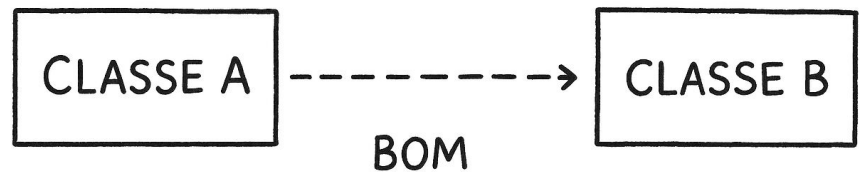
Arquitetura de software

Acoplamento: é o grau em que uma classe conhece a outra.

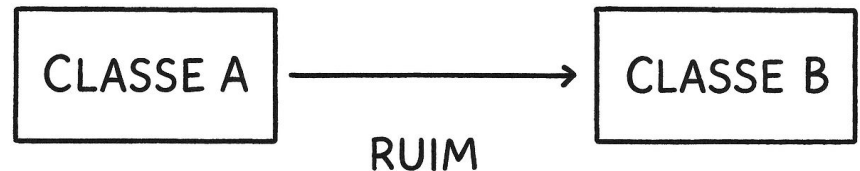
Se o conhecimento da classe A sobre a classe B for através de sua interface, temos um baixo acoplamento, e isso é bom.

Por outro lado, se a classe A depende de membros da classe B que não fazem parte da interface de B, então temos um alto acoplamento, o que é ruim.

BAIXO ACOPLAMENTO



ALTO ACOPLAMENTO



MODEL-VIEW-CONTROLLER

Arquitetura de software

```
// Interface define o contrato (a "interface pública" de ClasseB)
interface IClasseB {
    void executar();
}

// ClasseB implementa a interface
class ClasseB implements IClasseB {
    public void executar() {
        System.out.println("Executando ClasseB de forma controlada
pela interface!");
    }

    // Método interno não acessado por ClasseA
    private void detalheInterno() {
        System.out.println("Detalhe interno de ClasseB (ClasseA não
deve ver isso)");
    }
}
```

MODEL-VIEW-CONTROLLER

Arquitetura de software

```
// ClasseA depende apenas da interface
class ClasseA {
    private IClasseB classeB;

    public ClasseA(IClasseB classeB) {
        this.classeB = classeB;
    }

    public void processar() {
        System.out.println("ClasseA chamando ClasseB de forma
desacoplada...");
        classeB.executar();
    }
}
```

MODEL-VIEW-CONTROLLER

Arquitetura de software

```
// Teste
public class BaixoAcoplamento {
    public static void main(String[] args) {
        IClasseB b = new ClasseB();
        ClasseA a = new ClasseA(b);
        a.processar();
    }
}
```

ClasseA só precisa saber **que existe** um método **executar()** em **IClasseB**.

Se a implementação de **ClasseB** mudar, **ClasseA** não precisa ser modificada.

MODEL-VIEW-CONTROLLER

Arquitetura de software

```
//Exemplo de ALTO ACOPLAMENTO
// ClasseB sem interface
class ClasseB {
    public void executar() {
        System.out.println("Executando ClasseB...");
    }

    // Detalhe interno que não deveria ser acessado
    public void detalheInterno() {
        System.out.println("Executando detalhe interno de ClasseB!");
    }
}
```


MODEL-VIEW-CONTROLLER

Arquitetura de software

```
// ClasseA depende diretamente da implementação de ClasseB
class ClasseA {
    private ClasseB classeB = new ClasseB();

    public void processar() {
        System.out.println("ClasseA chamando ClasseB de forma
acoplada...");
        classeB.executar();

        // Aqui está o problema: acesso a um detalhe interno!
        classeB.detalheInterno();
    }
}
```

MODEL-VIEW-CONTROLLER

Arquitetura de software

```
// Teste
public class AltoAcoplamento {
    public static void main(String[] args) {
        ClasseA a = new ClasseA();
        a.processar();
    }
}
```

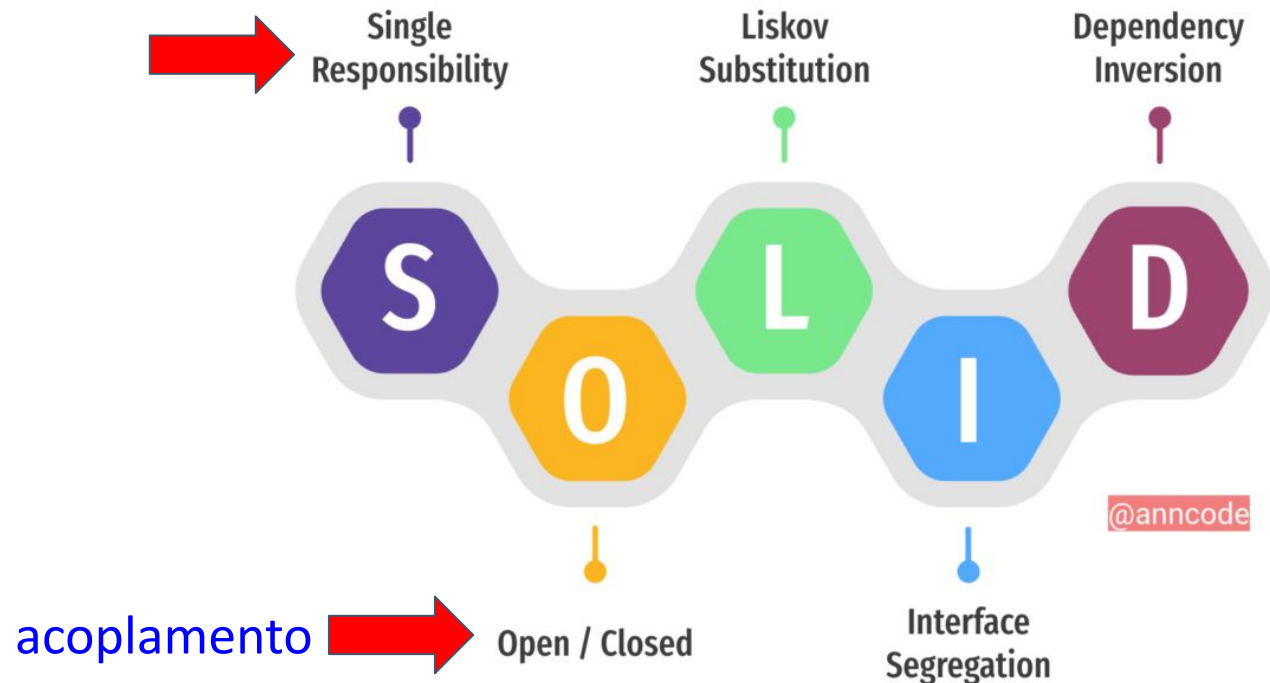
ClasseA depende fortemente da estrutura interna de **ClasseB**.

Se o método **detalheInterno()** mudar ou for removido, **ClasseA** quebra.

MODEL-VIEW-CONTROLLER

Arquitetura de software

Coesão: quando temos uma classe elaborada de forma que tenha um único e bem focado propósito, dizemos que ela tem uma alta coesão, e isso é bom. Quando temos uma classe com propósitos que não pertencem apenas a ela, temos uma baixa coesão, o que é ruim.



MODEL-VIEW-CONTROLLER

Arquitetura de software

- Na VIEW (camada de visualização/apresentação) há a interação com o usuário (entrada e saída)
- Em Java/Desktop (swing, awt, swt, **javafx** (FXML) etc.)
- Em Web (html, jsp, jsf, js, css, bootstrap, thymeleaf etc.), frontends/frameworks (react, angular, vue etc.)
- No CONTROLLER, as requisições do usuário são direcionadas pelo CONTROLLER para o MODEL. Se o MODEL gera respostas, envia para o CONTROLLER e este para a VIEW
- Com MVC, a manutenção e atualização se torna mais fácil; os testes podem ser isolados por camada (salvo testes de integração); requer do desenvolvedor maior atenção na etapa de modelagem e posterior implementação
- No MODEL, a comunicação com a base de dados pode utilizar a API Java Database Connectivity (JDBC)

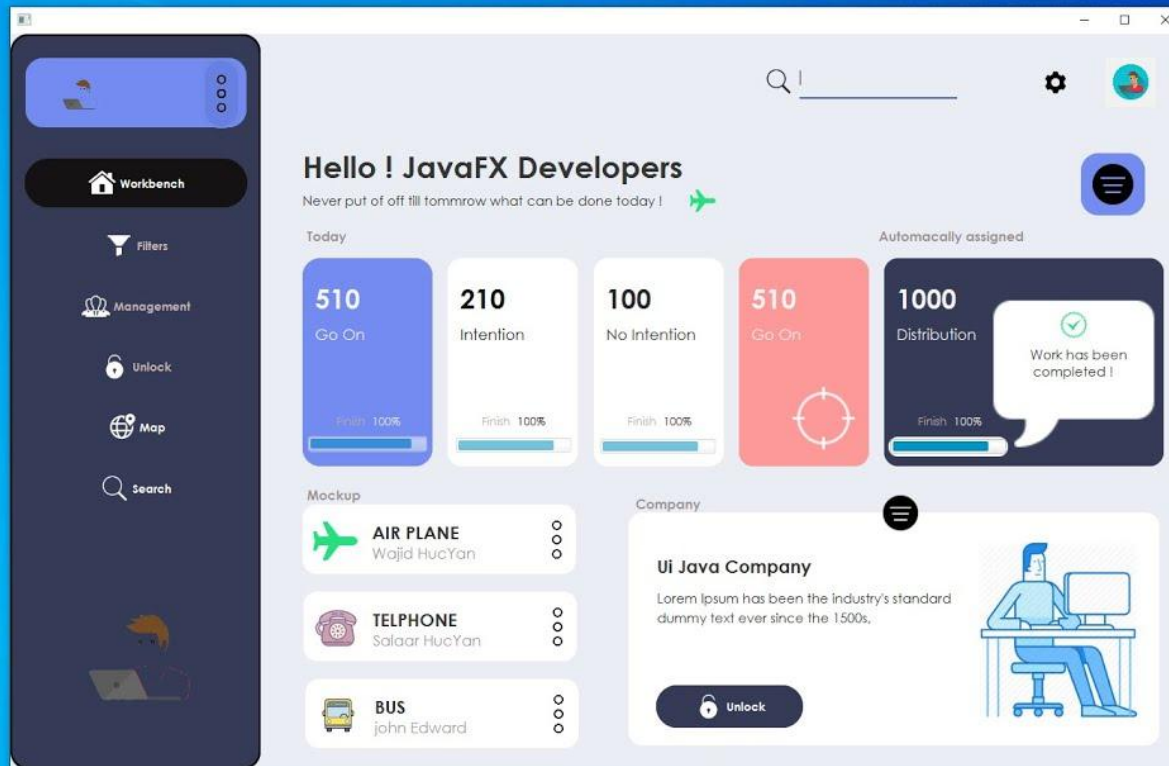
MODEL-VIEW-CONTROLLER

Arquitetura de software



MODEL-VIEW-CONTROLLER

Arquitetura de software



MODEL-VIEW-CONTROLLER

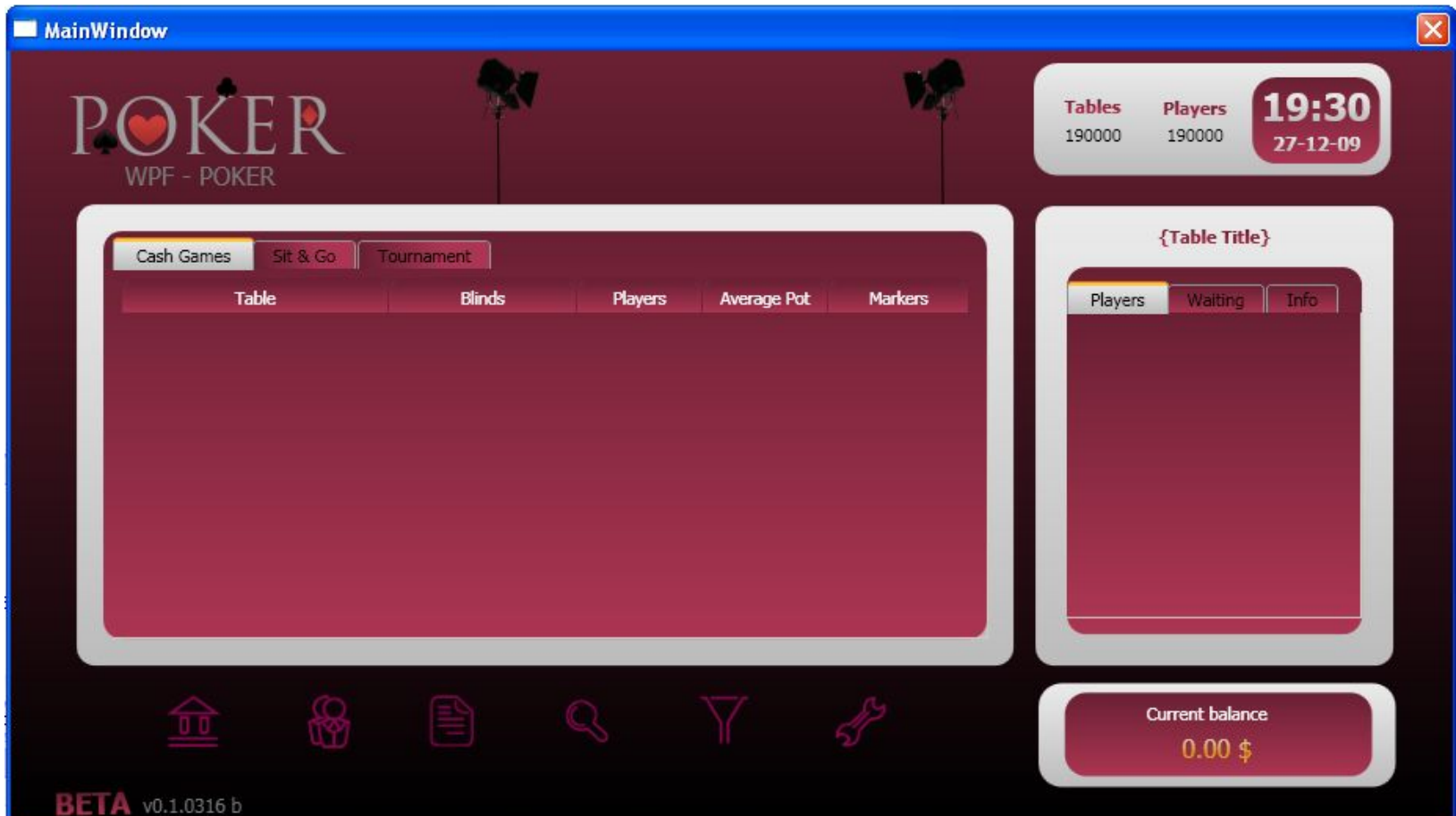
Arquitetura de software

JavaFX Design



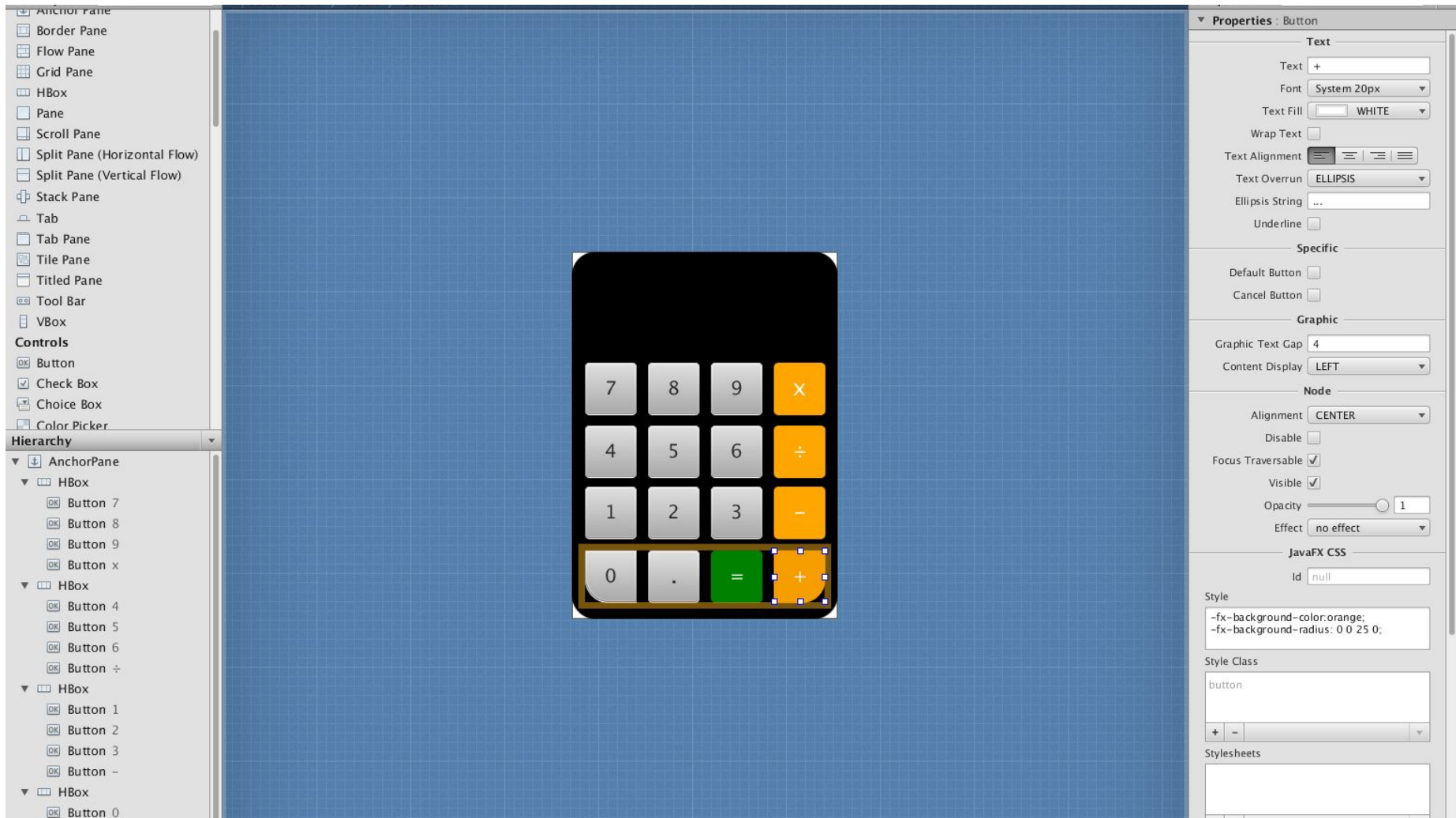
MODEL-VIEW-CONTROLLER

Arquitetura de software



MODEL-VIEW-CONTROLLER

Arquitetura de software



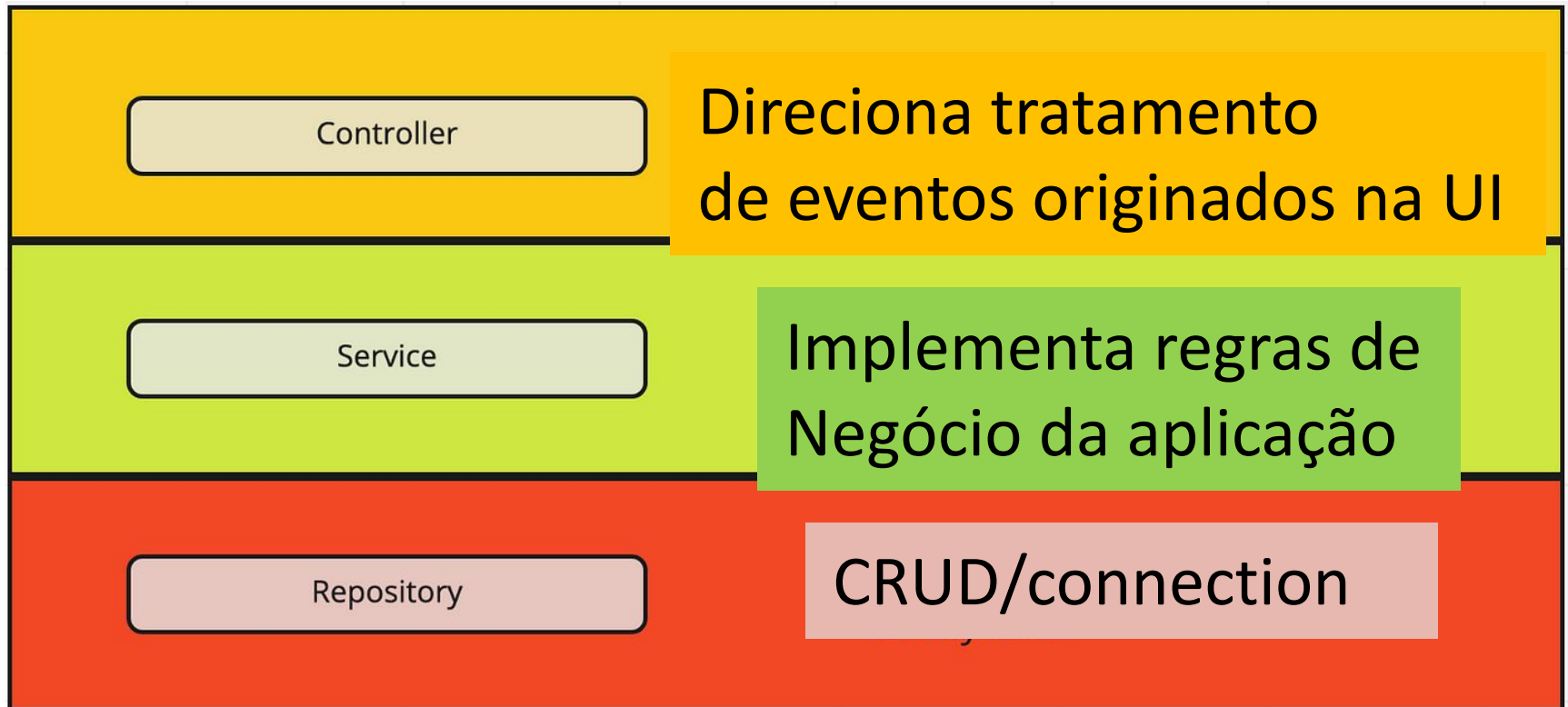
MODEL-VIEW-CONTROLLER

Arquitetura de software

- Cada TELA da UI é um arquivo .fxml, que pode ser criado na mão (hard coded) ou gerado por software ([SCENE BUILDER](#))
- Cada arquivo .fxml de TELA precisa definir sua classe CONTROLADORA, que trata os CONTROLES/ELEMENTOS da TELA

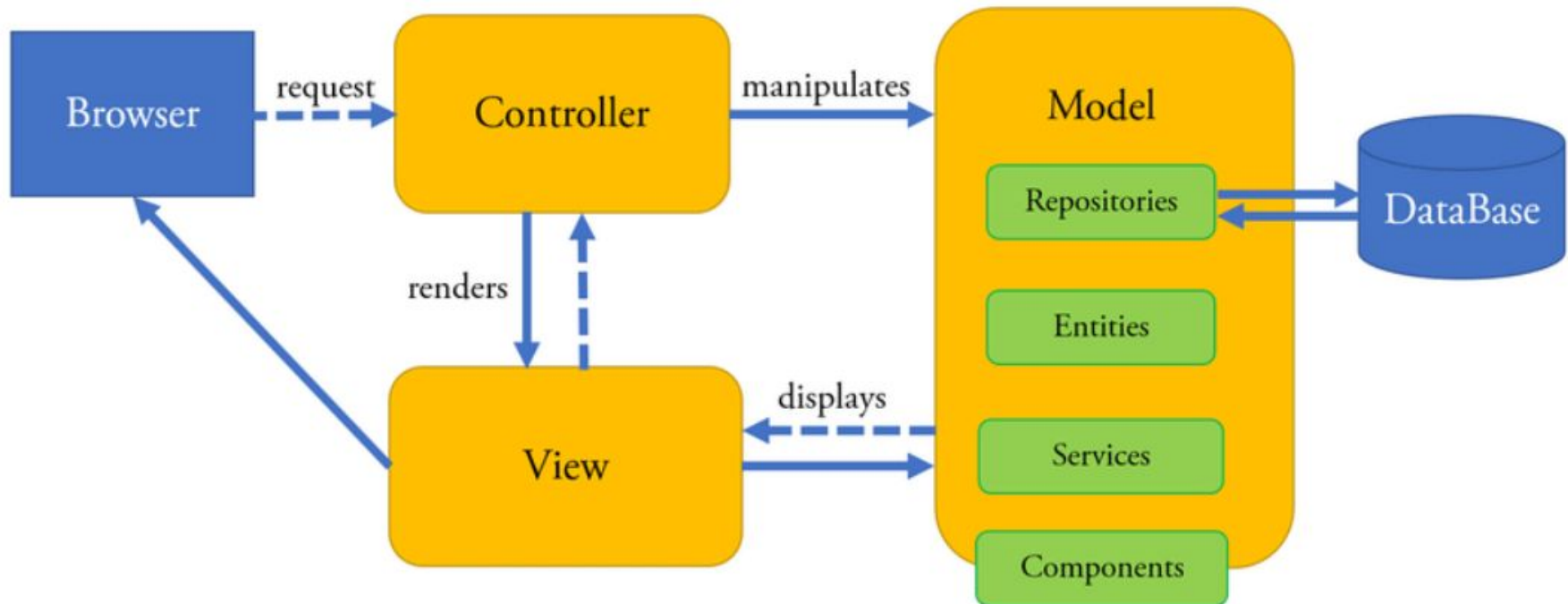
MODEL-VIEW-CONTROLLER

Arquitetura de software



MODEL-VIEW-CONTROLLER

Arquitetura de software



MODEL-VIEW-CONTROLLER

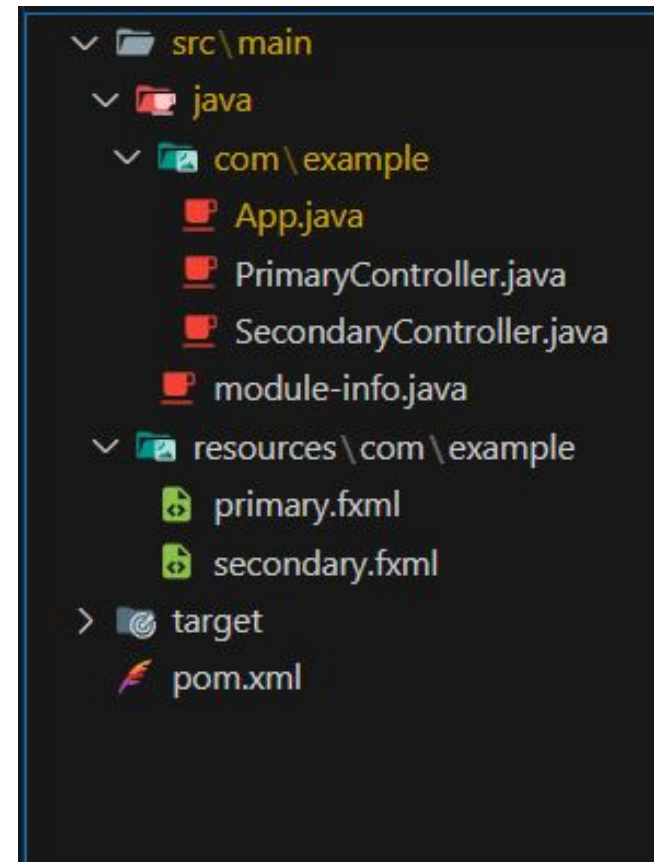
Arquitetura de software

- É uma boa prática criar ao menos mais três pacotes internos ao pacote MODEL
- **Entidades**: representa uma tabela do BD, suas referências e dependências. Em Java, uma classe JavaBean (POJO: Plain Old Java Objects) tem um construtor, atributos privados, getters e setters
- No pacote **REPOSITORY** colocamos classes relacionadas ao CRUD e acesso ao BD, para cada entidade.
- No pacote **SERVICE**, são classes com as regras de negócio. Acoplamento entre o pacote REPOSITORY e a classe de entidade.

MODEL-VIEW-CONTROLLER

Exemplo Introdutório
com JavaFX no VSCODE

- Ctrl + Shift + P: Create Java Project -> JavaFX (com Maven) – adiciona .jars das dependências
 - Group ID (com.example) – br.ufrn.tads *(este será o nome do package principal)*
 - Artifact ID (demo) – nome do projeto – test
 - Version 1.0 (pode variar)
 - *No src\main*
 - *\java*
 - *Group ID*
 - *\resources (telas do Scene Builder) – equivale a \view*
 - *\target (binários gerados)*
 - *module-info.java, pom.xml*



MODEL-VIEW-CONTROLLER

Exemplo Introdutório
com JavaFX no VSCODE

- *No src\main*
 - *\java*
 - *tecinfo\poo*
 - *controller*
 - *model*
 - *dao (repository)*
 - *entity*
 - *service*
 - *App.java*
 - *\resources (telas do Scene Builder) – equivale a \view*
 - *\target (binários gerados)*
 - *module-info.java, pom.xml*

